

EventHub

Security Assessment & Remediation Report

A microservices event-ticketing platform

Index Number	Name
IT22207272	Fonseka G N V S
IT22228062	Jayasinghe I A S A
IT22049872	Navod W D C
IT22246332	Hettiarachchi H A S L

Original project: github.com/wahallu/CTSE---Assignment

Modified project: see README for the new repository link

1. Introduction

EventHub is an event-ticketing web application implemented as five Node.js/Express microservices — *User*, *Event*, *Ticket*, *Payment* and an *API Gateway* — with a React (Vite) single-page front end and MongoDB storage. Users browse events, book seats, pay, and view their tickets; administrators manage events. The application is deployed to Azure Container Apps through a GitHub Actions CI/CD pipeline.

The original source (last commit before the semester start) was reviewed against the **OWASP Top 10 (2021)**. We identified **eleven distinct vulnerabilities** across authentication, access control, business logic, injection and configuration, and remediated all of them. Each fix is a separate commit with a detailed message, and a reproducible exploit harness demonstrates the before/after state.

2. Methodology and tools

- **Manual white-box review** of every controller, route and middleware.
- **Black-box dynamic testing** with a custom script (`security-tests/exploits.sh`) using `curl` and `jq` to send real attack requests through the gateway.
- **JWT forgery test** using Node's `crypto` to sign a token with the leaked default secret.
- **Dependency / secret review** of `package.json`, `docker-compose.yml` and the git history.

The harness runs 17 checks (V1–V10, several with sub-cases). The result was **17 vulnerable / 0 safe** on the original code and **0 vulnerable / 17 safe** after remediation.

3. Summary of findings

#	Vulnerability	OWASP 2021	Severity	Status
V1	Mass-assignment privilege escalation (register as admin)	A04 / A01	Critical	Fixed
V2	Hard-coded default JWT secret & committed DB credentials	A02 / A05	Critical	Fixed
V3	Broken access control / IDOR on user accounts	A01	Critical	Fixed
V4	Missing authentication on event management	A01	High	Fixed
V5	Client-controlled ticket price & status	A04	High	Fixed
V6	Client-controlled payment amount & status	A04	Critical	Fixed
V7	IDOR on tickets & payments	A01	High	Fixed
V8	NoSQL operator injection	A03	High	Fixed
V9	Security misconfiguration (CORS, headers, error leakage)	A05	Medium	Fixed
V10	Weak password policy & login brute force	A07	High	Fixed
V11	Race condition / seat overselling (TOCTOU)	A04	Medium	Fixed

4. Detailed findings and fixes

V1 — Mass-assignment privilege escalation Critical

Where: `userservice/src/controllers/userController.js` (`registerUser`).

Problem: the public registration endpoint copied the `role` field straight from the request body into the new user, so any anonymous person could create an administrator account.

```
// BEFORE
const { name, email, password, role } = req.body;
const user = await User.create({ name, email, password: hashedPassword, role });
```

Evidence: `POST /api/users/register {"role":"admin"}` → account created with `role=admin`.

Fix: the role is never taken from user input; self-registration is always a customer, and only an existing admin can change a role (through a whitelisted update). The first admin is created through a controlled bootstrap.

```
// AFTER — role is forced, not taken from the body
const user = await User.create({ name, email, password: hashedPassword, role: "customer" });
```

V2 — Hard-coded default JWT secret & committed credentials Critical

Where: gateway & user auth (`process.env.JWT_SECRET` || `"eventhub"`), `docker-compose.yml`.

Problem: the JWT signing secret defaulted to the literal string `"eventhub"`, and a live MongoDB Atlas connection string (`nisa1:1234@...`) was committed. With the known secret an attacker can forge a valid token for any user and role.

Evidence: a token signed with HMAC-SHA256 and key `eventhub` carrying `role:"admin"` was accepted by `GET /api/users` (HTTP 200).

Fix: a fail-fast `config/env.js` in every service refuses to start without a `JWT_SECRET` (min 32 chars) and `INTERNAL_API_KEY`; all secrets move to an untracked `.env` (documented in `.env.example`). The leaked database credentials must be **rotated in MongoDB Atlas** — deleting them from the file does not remove them from earlier commits.

V3 — Broken access control / IDOR on user accounts Critical

Where: `userservice/src/routes/userRoutes.js` & controller.

Problem: every authenticated user could `GET /api/users` (list all accounts including hashes metadata), and `PUT/DELETE /api/users/:id` on any id — allowing a customer to reset another user's password, delete accounts, or set their own role to admin.

Evidence: as a normal customer: list all users (200, 4 accounts); `PUT /api/users/<self> {"role":"admin"}` → became admin; `PUT /api/users/<bob> {"password":"..."}` → 200 (account takeover).

Fix: `GET /api/users` is admin-only; `:id` routes require `owner-or-admin` (`requireSelfOrAdmin`); role can only be changed by an admin; updates are field-whitelisted so `role` /other fields cannot be mass-assigned.

V4 — Missing authentication on event management High

Where: gateway routing + `eventservice` routes.

Problem: the gateway only authenticated `/api/users`, `/api/tickets` and `/api/payments`. All of `/api/events` (including create/update/delete) was unauthenticated, so anyone could create fake events or set a real event's seats to 0 to sabotage sales.

Evidence: anonymous `POST /api/events` → 201; customer `PUT /api/events/:id {"availableSeats":0}` → 200.

Fix: the gateway now protects every non-GET `/api/events` request; the Event Service requires an authenticated **admin** for create/update/delete, while reads remain public.

V5 — Client-controlled ticket price & status High

Where: `ticketservice/src/controllers/ticketController.js`.

Problem: `createTicket` trusted `price` and `status` from the request body, so a booking could be created for `price:0` and immediately marked `status:"booked"`.

Evidence: booking with `{"price":0,"status":"booked"}` stored a ticket with price 0, status booked.

Fix: price is computed server-side from the authoritative event price (`event.price × seatCount`) and status is forced to `pending` until payment completes. The owner id comes from the verified token, not the body.

V6 — Client-controlled payment amount & status Critical

Where: `paymentservice/src/controllers/paymentController.js`.

Problem: `createPayment` trusted `amount` and `status`, so a user could record a `completed` payment of `0.00` and receive a confirmed ticket without paying.

Evidence: `POST /api/payments {"amount":0,"status":"completed"}` created a completed 0.00 payment.

Fix: the amount is read from the verified ticket price and the status is set by the server; a user can only pay for their own ticket; a duplicate completed payment is rejected; the ticket is confirmed by the server after payment.

V7 — IDOR on tickets & payments High

Where: ticket & payment controllers/routes.

Problem: any user could read or cancel another user's tickets, list every ticket, and list every payment in the system.

Fix: list endpoints are scoped to the caller's own id (admins see all); per-record endpoints enforce owner-or-admin. Identity comes from the token.

V8 — NoSQL operator injection High

Where: `getAllTickets` query filter; all request inputs.

Problem: `?userId[$ne]=...` was passed straight into a Mongo query, returning every ticket regardless of owner (operator injection).

Fix: list filters use the trusted token identity instead of a client query parameter, and a shared `sanitize()` middleware recursively strips keys beginning with `$` or containing `.` from request bodies and params.

V9 — Security misconfiguration Medium

Where: all services.

Problem: `cors()` allowed any origin; no security headers were set; `X-Powered-By` advertised Express; error handlers echoed internal messages.

Fix: the gateway uses a CORS allow-list (`CORS_ORIGINS`); every service sets `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` and HSTS, disables `X-Powered-By`, and returns a generic message for 500s.

V10 — Weak passwords & login brute force High

Where: user registration; gateway rate limiting.

Problem: a password of `"1"` was accepted, and login allowed 100 attempts / 15 min — enough for credential stuffing.

Fix: a policy of ≥ 8 characters with letters and numbers is enforced (bcrypt cost 12); a strict limiter allows only 5 auth attempts / 15 min, and login returns a uniform response with a constant-time comparison to reduce enumeration.

V11 — Race condition / seat overselling Medium

Where: ticket $\rightarrow$ event seat update.

Problem: booking read `availableSeats` then wrote the decremented value in a separate request (time-of-check/time-of-use), so simultaneous bookings could oversell.

Fix: an atomic conditional update on the Event Service (`findOneAndUpdate({ availableSeats: { $gte: n } }, { $inc: { availableSeats: -n } })`) guarantees seats are only reserved when actually available; a failed ticket write rolls the reservation back.

5. Architectural hardening: the gateway trust boundary

A cross-cutting weakness was that the backend services had no notion of identity and could be called directly. We introduced a **single trust boundary at the API Gateway**:

- The gateway is the only component that verifies JWTs.
- It **strips** any client-supplied `x-internal-key` / `x-user-*` headers, then re-adds the shared `INTERNAL_API_KEY` and the verified identity.
- Each backend service rejects requests without a valid internal key (403) and reads identity/role from the gateway-set headers.

This closes header-spoofing and direct-access bypasses and gives every service a reliable identity for authorization.

6. New feature — Sign in with Google (OpenID Connect)

We added Google login using the **OpenID Connect Authorization Code flow with PKCE** (`userservice/src/controllers/oauthController.js`):

Control	Purpose
Authorization Code + PKCE (S256)	Client secret stays server-side; the code is bound to this client.
<code>state</code> parameter	Prevents login CSRF on the callback.
<code>nonce</code> in ID token	Prevents token replay.
ID-token validation (aud, exp, signature) + <code>email_verified</code>	Ensures the token is genuine and the email is owned.
Signed, httpOnly, SameSite cookie	Holds the short-lived flow secrets without a session store.
Token via URL fragment	App JWT is returned in <code>#token=...</code> so it is not sent to servers/logs.

New Google users are created as customers; an existing local account with the same verified email is linked. The endpoints return `503` when Google credentials are not configured, so the app still runs without them.

7. Vulnerabilities not fixed

All eleven identified vulnerabilities were fixed. A few *defence-in-depth* improvements are documented as future work rather than left as exploitable holes:

- **Real payment gateway**: payments are still simulated (marked completed server-side). Integrating a real PSP (e.g. Stripe) with webhook confirmation is out of scope for this assignment.
- **Refresh tokens / token revocation**: JWTs are stateless with a fixed expiry; a token blacklist or short-lived access + refresh tokens would allow immediate revocation.
- **Token storage in the browser**: the SPA keeps the JWT in `localStorage`; moving to httpOnly cookies would remove XSS token-theft risk but requires CSRF protection and was kept compatible with the existing client.
- **Automated dependency scanning & SAST in CI** (see best practices).

8. Best practices that would have prevented these issues

- **Secrets management**: never commit secrets; use `.env` / secret stores (Azure Key Vault) and pre-commit secret scanning; fail fast on missing config.
- **Secure defaults & least privilege**: deny by default, whitelist inputs, and never derive privileges (role) from client input.
- **Server-authoritative business logic**: price, amount, status and identity must be decided server-side, never trusted from the client.
- **Centralised authN/authZ**: a single verified trust boundary with per-route authorization checks.
- **Input validation & output encoding**: validate types, reject Mongo operators, and never echo internal errors.
- **Security in CI/CD**: add `npm audit` / Dependabot, SAST (the project already runs SonarCloud), and secret scanning to the existing GitHub Actions pipeline; enforce code review.

- **Threat modelling & security testing:** a repeatable exploit suite (as added here) run in CI catches regressions.

9. Conclusion

The original EventHub contained eleven distinct, largely critical vulnerabilities spanning authentication, access control, business logic, injection and configuration. All were remediated with focused, individually-committed changes and verified by an automated exploit harness that moved the application from **17 vulnerable / 0 safe** to **0 vulnerable / 17 safe**. A Google OpenID Connect login was added following current OAuth 2.0 security best practice. The remaining items are defence-in-depth enhancements documented for future work.